Parcelo Phase 1: Build Prompt

Start here: what this file is, in plain English

What it is. The long text below the line is a *prompt*: a detailed brief that you paste into an AI coding assistant (for example Claude Code) so it builds Phase 1 of Parcelo for you. Think of it as the written instructions you'd give a builder before they start work. You don't have to understand every technical word; the assistant does.

What Phase 1 is. The first, smallest working version of Parcelo:

- A customer signs up, enters pickup and drop-off, sees the price, and books.
- They get a tracking number.
- An admin updates the delivery status and uploads a proof-of-delivery photo.
- Anyone can type a tracking number and see where the package is.

Drivers, dispatching, stock and text messages come in later phases, so the prompt tells the assistant not to build them yet.

What each numbered section of the prompt does

Section	In plain words
1. What Phase 1 delivers	The goal, and a "do not build" list so the assistant doesn't wander off.
2. Tech stack	The tools to use (React for the screens, Firebase for the backend). Fixed so the assistant doesn't pick something else.
3. Repository structure	How to organise the folders: a customer app, an admin app, a shared folder, and the backend.
4. Design system	The look: dark background, teal buttons, status colours, fonts, spacing.
5. Data model	What information is stored (users, quotes, orders, shipments, tracking records) and what each holds.
6. Business rules	The rules of the business: how the price is worked out, which status moves are allowed, when a photo is needed, how tracking numbers are made.
7. Screens	Every page in the customer app and the admin app, and what it shows.
8. Cloud Functions	The backend "workers" that do the important actions safely.
9. Security requirements	The safety rules from the Secure Coding file, summarised so the assistant must follow them.
10. Build order	11 small steps. The assistant stops after each one so you can check it.
11. Acceptance checks	A checklist that proves Phase 1 works.
12. How to work	How the assistant should behave: ask if unsure, keep it simple, don't add extras.

Words you may not know

Word	Meaning
Prompt	The instructions you give to an AI assistant.
Repository (repo)	The project folder that holds all your code.
Monorepo	One repo that holds several apps together (here: customer app, admin app, shared code, backend).
React / Vite	Tools for building websites. React builds the screens; Vite runs and packages them.
Firebase	Google's ready-made backend: login, database, file storage, hosting, and server code, so you don't run your own server.
Cloud Function	A small backend program that does protected actions (booking, status changes).
Security rules	A rulebook on the database saying who can read or write what.

Custom claims	The role label (customer/admin) stored safely in a user's login.
Emulator	A copy of Firebase running on your own computer for safe testing.
Quote	A price offer valid for 15 minutes.
Transaction	Several saves done all-or-nothing.
Deploy	Publishing the finished apps to the internet.

How to use it, step by step

1. Create an empty folder for the project and open it in your coding assistant.
2. Copy everything below the line (from "You are a senior full-stack engineer" to the end) and paste it as your first message.
3. If your assistant lets you attach files, also attach your Parcelo blueprint PDF and the Secure Coding file.
4. The assistant does step 1 and stops. Read its summary, run what it tells you to run, and check it works.
5. Type "continue to step 2" and repeat until step 11.
6. When the acceptance checks in section 11 pass, Phase 1 is done.

Tips

- Don't skip the checking. The step-by-step stops are there so a mistake in step 3 doesn't get buried under steps 4 to 11.
- If the assistant says something you don't understand, ask it: "explain that in simple words".
- You'll need a free Firebase account. Cloud Functions need the pay-as-you-go (Blaze) plan, which requires a payment card, but the free monthly allowance normally covers a project this size.

> Paste everything below the line into your coding assistant (e.g. Claude Code) at the root of an empty repository. If your tool lets you attach files, also attach the Parcelo Architecture and Design Blueprint (v2.0) and `Parcelo_Secure_Coding_Practices.md`.

You are a senior full-stack engineer. Build **Phase 1 of Parcelo**, a logistics platform, using React + Vite and a single Firebase backend. Work in small, verifiable steps, and stop for my review at the end of each numbered step in section 8.

1. What Phase 1 delivers

A customer can register, book a delivery (3 steps), see an instant price, get a tracking number and follow the package. An admin runs everything by hand from a simple dashboard. The public can look up a tracking number with no login. Phase 1 must work on its own.

Not in Phase 1 (do not build): driver app, dispatcher role UI, assignment, live location, inventory, notifications, reports, online payments.

2. Tech stack (fixed)

- React + Vite, React Router, React Context for auth state, Tailwind CSS with CSS variables, lucide-react icons, Leaflet + OpenStreetMap, Nominatim for geocoding (on button press only, never per keystroke)
- Firebase: Authentication (email/password), Cloud Firestore, Cloud Storage, Cloud Functions (Node.js, callable), security rules, Hosting (one site per app), App Check
- Roles via Firebase custom claims: `customer`, `admin` (dispatcher and driver exist as constants only)
- Validation in functions with Zod; tests with Vitest and the Firebase Emulator

3. Repository structure

```
parcelo/
  apps/
    customer/   (Customer app + public tracking page)
    ops/        (Ops dashboard, admin only in Phase 1)
  packages/
    shared/     (firebase init, auth, components, constants, utils, theme)
  firebase/
    functions/src/{booking,shipments,staff,utils}/ + index.js
    firestore.rules  storage.rules  firestore.indexes.json
  firebase.json  .firebaserc  package.json (workspaces)  README.md
```

Each app has: `index.html`, `vite.config.js`, `.env`, `public/`, `src/{main.jsx, App.jsx, layouts/, pages/, components/, hooks/, api/}`. Apps never call each other; they share the Firebase project and `@parcelo/shared`.

4. Design system (implement in `packages/shared/src/theme/tokens.css` and a Tailwind preset)

Dark dashboard theme:

- bg `#0F172A`, surface `#1E293B`, border `#334155`, accent `#14B8A6` (hover `#2DD4BF`, pressed `#0D9488`), text `#F1F5F9`, muted `#94A3B8`
- Status colours: Pending `#F59E0B`, In Transit `#3B82F6`, Out for Delivery `#8B5CF6`, Delivered `#22C55E`, Failed `#EF4444`. Badges always show the word, pill-shaped.
- Fonts (self-hosted in shared/theme, weights 400/500/600/700, `font-display: swap`, system fallbacks): Space Grotesk for headings, IBM Plex Sans for body, IBM Plex Mono for tracking numbers/IDs
- Radius: cards 12px, controls 8px. Borders 1px, no shadows. Focus ring 2px `#2DD4BF` with 2px gap. Buttons 44px high. Breakpoints 360/768/1024, design phone-first. Every list has skeleton, empty and error (with Retry) states.
- Button wording starts with a verb ("Confirm booking"). Error messages say what to do next.

Shared components: Button, Input, Select, Card, StatusBadge, Table, Modal, Toast, Tabs, Timeline, Stepper, MapView, Skeleton, EmptyState, ErrorState, RoleGuard, WrongAppNotice.

5. Data model (Firestore)

- `users/{uid}`: name, email, phone, role (display copy only; authorization uses the custom claim)
- `quotes/{id}`: customerId, pickup, dropoff (text + lat/lng), weightKg, size, distanceKm, price, expiresAt, used
- `orders/{id}`: customerId, customerName, pickup/dropoff addresses, price, paymentMethod (`pay_on_delivery` | `bank_transfer`), paymentStatus (`unpaid` | `pending_transfer` | `paid`), status (`placed` | `confirmed` | `shipped` | `delivered` | `cancelled`), createdAt
- `shipments/{id}`: orderId, customerId, trackingNumber, status, riderName, riderPhone, proofPhotoPath, deliveredAt, failureReason; subcollection `events/{id}`: status, note, changedBy, createdAt (append-only)
- `publicTracking/{trackingNumber}`: status, destinationArea, timeline[] only. No names, phones, full addresses or internal ids.
- `settings/pricing`: baseFee 1500, perKm 120, perKg 100, sizeFactors {small 1.0, medium 1.2, large 1.5}

6. Business rules

- Shipment statuses: Pending -> Assigned -> In Transit -> Out for Delivery -> Delivered; Failed (from In Transit/Out for Delivery, needs a reason, can retry to Out for Delivery); Cancelled only while Pending. In Phase 1 admin moves Pending -> In Transit directly (Assigned is disabled until Phase 2). Keep the rules in one pure module `statusRules.js` used by both shared and functions (same logic, with a test that guards they match).
- Price = (baseFee + perKm x distanceKm + perKg x weightKg) x sizeFactor, rounded to nearest N50. Test: 12 km, 3 kg, medium = N3,900. Distance = straight-line (haversine) x 1.3.
- Quote expires after 15 minutes, belongs to the caller, is single-use. Booked price never changes.
- Delivered requires a proof photo. Failed requires a reason. Customers may cancel only while Pending.
- Tracking numbers are created server-side with `crypto.randomBytes`, format `TRK-` + 10 characters from `ABCDEFGHJKLMNPQRSTUVWXYZ23456789`. Use one canonical format everywhere, validate with a regex, uppercase and trim before lookup.

7. Screens

Customer app: `/` (tracking box + Book button), `/track/:trackingNumber` (public timeline, delivery photo when Delivered, friendly not-found), `/login`, `/register`, `/book` (3 steps: pickup/drop-off with "Find on map" buttons; package weight and size; quote + payment method + Confirm booking, with success screen and copyable tracking number), `/orders`, `/orders/:id` (price, status timeline, rider contact if present).

Ops dashboard (admin): `/login`, `/` (count cards: Pending, In Transit, Out for Delivery, Delivered, Failed), `/orders` (search by tracking number, filter by status), `/shipments/:id` (next-status button showing only valid moves, rider name/phone form, proof upload with client resize to ~1280px wide, payment status select, status history).

Sidebar is built from role (admin sees all links). Wrong-app login shows WrongAppNotice with a link to the right app.

8. Cloud Functions (callable; all protected actions)

`getQuote`, `bookDelivery`, `cancelOrder`, `setPaymentStatus`, `updateShipmentStatus`, `saveProof`, plus a customer-only registration path (role always `customer`). `bookDelivery` creates order + shipment + first event + publicTracking record in **one transaction**.

9. Security requirements (non-negotiable; follow `Parcelo_Secure_Coding_Practices.md`)

- Firestore and Storage rules deny by default. No client writes to orders, quotes, shipments status, paymentStatus or publicTracking. Admin may write only `riderName` / `riderPhone` on a shipment (use `affectedKeys().hasOnly`), with type and length checks. `publicTracking` allows `get` only.
- Every function: authenticate, check role, validate input with Zod (reject unknown fields, bound numbers), check state, write in a transaction, return minimal data and generic errors using standard `HttpsError` codes.
- `bookDelivery` recomputes the price server-side, verifies the quote belongs to the caller, is unexpired and unused.
- Roles come only from custom claims set by the Admin SDK. Never read a role from request data. The first admin is created manually.
- Storage: `proofs/{shipmentId}/...`, admin only, JPG/PNG, under 2 MB, random filenames, strip EXIF on the client.
- No secrets in `VITE_*` variables. Enable App Check. Add security headers and a CSP in `firebase.json` for both sites.
- No `dangerouslySetInnerHTML`. Validate coordinates and all user input.

10. Build order (stop after each step and summarise what changed, how to run it, and how I can verify it)

1. Monorepo scaffold, Firebase config, both apps booting with the shared dark theme and fonts
2. Shared components, AuthContext, layouts, RoleGuard, WrongAppNotice, with a test page showing every component
3. Auth: register, login, custom claims, role-based landing; document how to create the first admin
4. `getQuote` function + unit test (12 km, 3 kg, medium = 3900)
5. `bookDelivery` + the 3-step booking pages
6. Firestore and Storage rules + emulator tests (a customer cannot read another customer's order; no client writes; cannot list `publicTracking`; cannot escalate role)
7. My orders and Order details
8. Ops dashboard: Dashboard, Orders, Shipment page with `updateShipmentStatus`
9. Public tracking page
10. `saveProof` + upload + payment status; Delivered refused without a photo
11. Loading/empty/error states, phone-size testing, README, deploy both apps to Firebase Hosting

11. Acceptance checks (all must pass)

- New customer can register, book, and receive a tracking number; price on screen equals the formula
- A customer cannot open another customer's order
- Admin can move status forward only along valid steps; skipping is refused with a clear message
- Delivered is refused without a photo; Failed requires a reason
- Public tracking shows timeline only, no personal data; unknown numbers show a friendly message
- Booking and tracking work at 360px width
- Emulator rule tests and function unit tests pass; `npm audit` reviewed

12. How to work

- Ask me before assuming anything not in this prompt; otherwise pick the simplest option and note it.
- Keep files small and named as in section 3. Prefer plain, readable code with short comments on security-relevant lines.
- Do not add features, libraries or phases beyond what is listed. If something conflicts with the blueprint, tell me and propose a fix.
- Give me exact commands to run (install, emulators, tests, deploy).

Start with step 1 now.